PROJECT REPORT · BAEIE102

Sensors and Signal Conditioning

IoT-Enabled Motor Predictive Maintenance System Using Vibration Signature Analysis

Dual-sensor vibration monitoring and condition assessment with an ESP32

SUBMITTED BY	Lakshmi A
REGISTRATION NUMBER	25BEI0048
FACULTY	Dr. S. Vivekanandan
DEPARTMENT	Electronics and Instrumentation Engineering
PROJECT TYPE	Individual (solo) project
DATE	November 2026

Abstract

Unplanned failure of rotating machinery is a major source of downtime in industry, and most such failures announce themselves through changes in vibration long before they become visible or audible. This project presents a low-cost, IoT-enabled predictive maintenance prototype that monitors the vibration of a 775 DC motor and assesses its operating condition in real time.

Two sensing paths are used. A piezoelectric disc bonded to the motor body feeds an op-amp based signal-conditioning stage and the 12-bit ADC of an ESP32, which samples it at 1 kHz in blocks of 128 samples. An MPU6050 three-axis MEMS accelerometer, connected on a separate I^2C bus, supplies a complementary acceleration measurement. The firmware removes the DC offset, computes the vibration RMS, smooths it with a five-point moving average, and applies a Hamming-windowed FFT to extract the dominant frequency and peak spectral magnitude. The three-axis acceleration RMS is computed from the MPU6050 after removing the mean (gravity and offset).

The machine condition is classified as LOW SIGNAL, WARNING or ALERT using RMS thresholds of 6.0 and 10.0 ADC counts with hysteresis to prevent chattering. The state is shown through green and red LEDs, a buzzer with a push-button mute, and an SSD1306 OLED display. Every 20 seconds, five parameters are uploaded over Wi-Fi to a ThingSpeak channel for remote monitoring and trend logging.

Bench tests show correct state transitions, correct hysteresis behaviour in the serial log, and cloud-logged excursions of the vibration RMS from a baseline of roughly 2 to 5 counts up to about 20 counts. The system provides a dependable, affordable foundation for condition-based maintenance, with calibration against known fault conditions and relay-based motor cut-off identified as the next steps.

Keywords: *predictive maintenance, vibration analysis, piezoelectric sensor, MPU6050, FFT, ESP32, ThingSpeak, signal conditioning*

Acknowledgements

I would like to express my sincere gratitude to Dr. S. Vivekanandan, faculty for BAEIE102 — Sensors and Signal Conditioning, for his guidance, for the clarity of the course material on sensors and signal conditioning, and for the encouragement to build a working system rather than a purely theoretical one.

I am thankful to the Department of Electronics and Instrumentation Engineering for providing the laboratory facilities and components used in this project. I also thank my classmates and friends for their suggestions during testing, and my family for their constant support throughout the semester.

Lakshmi A (25BEI0048)

Table of Contents

Abstract . . . 2
Acknowledgements . . . 2
Table of Contents . . . 3
List of Figures . . . 4
List of Tables . . . 4
List of Abbreviations . . . 4
1. Introduction . . . 5
1.1 Background . . . 5
1.2 Problem statement . . . 5
1.3 Objectives . . . 5
1.4 Scope and novelty . . . 5
1.5 Organisation of the report . . . 5
2. Methodology . . . 6
2.1 Overall approach . . . 6
2.2 Hardware components . . . 6
2.3 Vibration sensing principles . . . 7
2.4 Signal conditioning of the piezoelectric sensor . . . 7
2.5 Digital signal processing . . . 7
2.6 Machine condition classification . . . 7
2.7 Alerting and IoT reporting . . . 7
2.8 Design evolution from the proposal . . . 7
3. Implementation . . . 8
3.1 Hardware assembly . . . 8
3.2 Pin and bus assignment . . . 8
3.3 Firmware architecture . . . 9
3.4 Signal-processing parameters . . . 9
3.5 Status logic and output control . . . 9
3.6 OLED display . . . 10
3.7 ThingSpeak configuration . . . 10
4. Results and Discussion . . . 10
4.1 Test setup . . . 10
4.2 Normal operation . . . 11
4.3 Elevated vibration and alert operation . . . 11
4.4 Serial Monitor log and hysteresis verification . . . 11
4.5 ThingSpeak remote monitoring . . . 13
4.6 Discussion and limitations . . . 13
5. Conclusion . . . 14
6. Future Scope . . . 14
Bibliography . . . 15

List of Figures

Figure 2.1: System block diagram 6
Figure 3.1: Assembled prototype in normal operation (LOW SIGNAL, green LED) 8
Figure 3.2: Top view of the prototype showing sensor mounting 8
Figure 4.1: Prototype in the ALERT state (red LED) 11
Figure 4.2: Arduino IDE Serial Monitor output 12
Figure 4.3: ThingSpeak dashboard showing Fields 1 to 4 13

List of Tables

Table 2.1: Hardware components and their functions 6
Table 2.2: Proposed design versus implemented design 8
Table 3.1: ESP32 pin and bus assignment 9
Table 3.2: Signal-processing parameters 9
Table 3.3: Status thresholds and hysteresis 10
Table 3.4: Output indication for each machine state 10
Table 3.5: ThingSpeak field mapping 10
Table 4.1: Serial Monitor sample demonstrating hysteresis 12
Table 4.2: OLED readings from the prototype photographs 11
Table 4.3: Summary of ThingSpeak observations 13

List of Abbreviations

ADC	Analog-to-Digital Converter	**IoT**	Internet of Things
DC	Direct Current	**LED**	Light-Emitting Diode
DLPF	Digital Low-Pass Filter	**MEMS**	Micro-Electro-Mechanical System
FFT	Fast Fourier Transform	**OLED**	Organic Light-Emitting Diode
GPIO	General-Purpose Input/Output	**RMS**	Root Mean Square
I^2C	Inter-Integrated Circuit	**SDA / SCL**	I^2C serial data / serial clock lines

1. Introduction

1.1 Background

Rotating machinery such as motors, pumps, fans and compressors forms the backbone of industrial drives and everyday utilities. Maintenance of such equipment is commonly organised in three ways: reactive maintenance (repair after failure), preventive maintenance (service at fixed intervals) and predictive maintenance (service when measured condition indicates it is needed) [2]. Predictive maintenance avoids both unexpected breakdowns and unnecessary servicing, because the decision to intervene is based on the actual health of the machine.

Vibration is one of the most informative condition indicators for rotating machines. Faults such as bearing wear, shaft imbalance and misalignment change the amplitude and the frequency content of the vibration well before the fault produces an audible or visible symptom [1]. Measuring and analysing vibration therefore gives early warning of developing faults.

1.2 Problem statement

Industrial condition-monitoring systems are effective but expensive. At the other end of the scale, the inexpensive vibration modules commonly used in student and hobby projects act as simple switches that report only whether vibration is present, discarding the amplitude and frequency information that makes diagnosis possible. There is a need for an affordable system that quantifies vibration, analyses its frequency content, signals the machine condition locally and reports it remotely.

1.3 Objectives

- To develop an embedded system for monitoring motor vibration.
- To acquire vibration signals using a piezoelectric sensor with an appropriate signal-conditioning stage.
- To measure three-axis acceleration using an MPU6050 accelerometer.
- To process the data on an ESP32 and extract meaningful parameters: vibration RMS, dominant frequency, peak spectral magnitude and acceleration RMS.
- To present real-time operating information on an OLED display.
- To implement visual and audible alerts for different vibration conditions.
- To enable remote monitoring through the ThingSpeak IoT platform.

1.4 Scope and novelty

The project is a bench-scale prototype in which a 775 DC motor represents the machine being monitored. Its novelty lies in three features:

- **On-board frequency-domain analysis:** the ESP32 computes an FFT of the vibration signal rather than relying only on a magnitude threshold, so the frequency signature is available alongside the amplitude.
- **Two sensing principles in one system:** a piezoelectric disc with its own signal-conditioning stage, and a MEMS accelerometer, demonstrating two different sensor interfaces.
- **A complete chain at low cost:** sense, condition, process, indicate and report, mirroring the structure of industrial condition-monitoring systems.

The alert thresholds used are provisional values chosen from bench observation, expressed in raw ADC counts; they are not calibrated against known fault conditions (see Section 4.6).

1.5 Organisation of the report

Chapter 2 describes the design approach, components, sensing principles and signal processing. Chapter 3 documents the hardware assembly and firmware. Chapter 4 presents and discusses the test results. Chapters 5 and 6 give the conclusion and future scope, followed by the bibliography.

2. Methodology

2.1 Overall approach

The system follows a sense, condition, acquire, process, decide, indicate and report chain, shown in Figure 2.1. The motor vibration is picked up by the piezoelectric disc and the MPU6050. The piezo signal is conditioned by an analog stage and digitised by the ESP32 ADC; the MPU6050 delivers digital samples over I^2C. The ESP32 computes the vibration parameters, classifies the machine state, drives the local indicators and uploads the data to ThingSpeak.

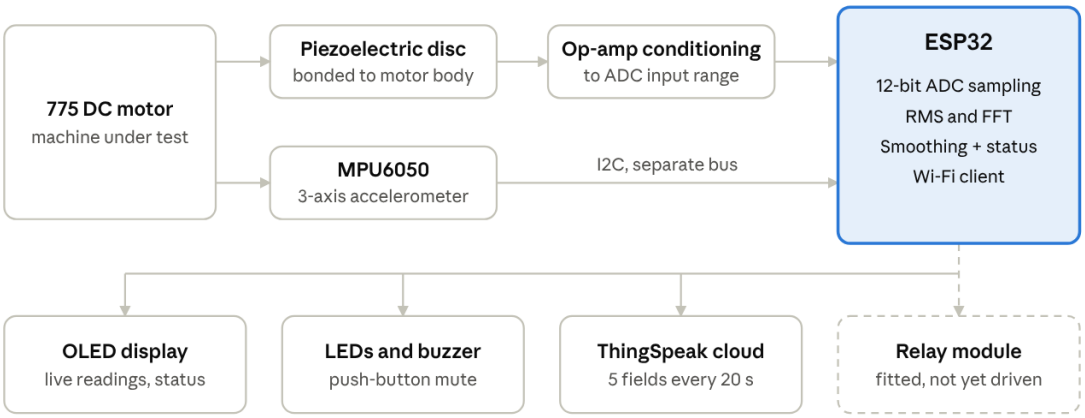


Figure 2.1. System block diagram: two sensors, one controller and four outputs. The dashed lines mark the relay, which is installed but not yet driven by the firmware.

2.2 Hardware components

Table 2.1 lists the components used and the role of each.

Table 2.1: Hardware components and their functions

Component	Function
ESP32 development board	Main controller: sensor acquisition, signal processing, status logic and Wi-Fi communication
Piezoelectric disc	Detects mechanical vibration and converts it into an electrical signal
Op-amp signal-conditioning stage with passive components	Conditions the high-impedance piezo signal for the ESP32 ADC
MPU6050 accelerometer	Measures acceleration along the X, Y and Z axes (I^2C)
775 DC motor	Represents the machine being monitored
SSD1306 OLED display (128 × 64, I^2C)	Shows measurements and machine status locally
Green and red LEDs	Visual indication of operating condition
Active buzzer	Audible warning in the ALERT state
Push button	Mutes or un-mutes the buzzer
Relay module	Electrically controlled switching interface, fitted in the prototype for motor-power control
12 V DC adapter	Supplies the motor
Breadboard, resistors, capacitors, jumper wires	Circuit assembly

2.3 Vibration sensing principles

Piezoelectric sensing. A piezoelectric disc generates an electrical charge when it is mechanically strained, so a disc fixed to the motor body produces a signal that follows the motor's vibration. The disc behaves as a high-impedance, AC-type source, which is why it cannot be connected directly to a microcontroller input without conditioning [7].

MEMS accelerometer sensing. The MPU6050 contains a three-axis MEMS accelerometer with an internal digital converter and an I^2C interface [3]. In this project the ±2 g range and a 94 Hz internal digital low-pass filter are selected; only the accelerometer is used, and the gyroscope is left unused.

2.4 Signal conditioning of the piezoelectric sensor

The ESP32 ADC accepts only a unipolar input between 0 V and the 3.3 V supply [4], whereas a piezoelectric disc produces a small signal that swings about its own zero level. The conditioning stage, built around an operational amplifier on the breadboard, buffers the high-impedance source, provides gain and places the signal within the ADC input range, using standard op-amp techniques [8]. Because the conditioned signal rides on a DC level, the firmware removes the mean of every block before computing RMS and the FFT.

2.5 Digital signal processing

The ESP32 processes the data in repeating cycles, using standard sampling, RMS and spectral-analysis methods covered in the course [11], as follows.

Sampling. The piezo signal is sampled at an intended 1 kHz in blocks of N = 128 samples, giving a 128 ms observation window. The Nyquist frequency is 500 Hz.

Mean removal. The block mean is subtracted from every sample to remove the DC component.

RMS. The vibration level is RMS = $\sqrt{[(1/N)\ \Sigma\ (x_i - \bar{x})^2]}$, expressed in ADC counts.

Smoothing. The RMS of the last five blocks is averaged (moving average) to suppress isolated spikes, giving the filtered RMS used for classification.

FFT. A Hamming window is applied, the FFT is computed and converted to magnitudes [6]. The strongest bin between 7.8 Hz and about 484 Hz gives the dominant frequency and the peak magnitude. With f_s = 1000 Hz and N = 128 the frequency resolution is f_s / N = 7.8125 Hz, so reported frequencies are multiples of 7.8125 Hz (for example 414.1 Hz is bin 53).

Acceleration RMS. The MPU6050 is read 128 times at 100 Hz. The mean of each axis is removed (eliminating gravity and offset) and the combined RMS is $\sqrt{[(1/N)\ \Sigma\ (a_x^2 + a_y^2 + a_z^2)]}$ in m/s^2, using the mean-removed values.

2.6 Machine condition classification

The filtered piezo RMS determines one of three states: LOW SIGNAL (vibration within the lower range), WARNING (warning threshold crossed) and ALERT (higher threshold crossed). A simple threshold comparison would make the state flicker whenever the RMS hovers near a limit, so hysteresis is used: the thresholds for entering a state are higher than those for leaving it. The values are given in Table 3.3.

2.7 Alerting and IoT reporting

The state drives a green LED (LOW SIGNAL), a red LED (WARNING and ALERT) and a buzzer (ALERT only, unless muted with the push button). The ESP32 joins the Wi-Fi network in station mode and writes the measured parameters to a ThingSpeak channel at fixed intervals, enabling remote viewing of trends [5].

2.8 Design evolution from the proposal

The design was refined during development. Table 2.2 compares the original proposal with the system as built.

Table 2.2: Proposed design versus implemented design

Aspect	Proposed	Implemented
Accelerometer	ADXL345 [12]	MPU6050 three-axis accelerometer
Local display	16 × 2 LCD	128 × 64 SSD1306 OLED
Piezo conditioning	Op-amp charge amplifier (TL072 / LM358)	Op-amp conditioning stage feeding the ESP32 ADC
Fault decision	Comparison of the FFT spectrum with a stored healthy baseline	Smoothed RMS thresholds with hysteresis; FFT frequency and peak magnitude computed and logged
Fault response	Relay cuts motor power and buzzer sounds	LEDs and mutable buzzer; relay module installed, firmware cut-off not yet enabled
Cloud platform	ThingSpeak or Blynk	ThingSpeak, five fields

3. Implementation

3.1 Hardware assembly

The assembled prototype is shown in Figure 3.1. The 775 DC motor, powered from a 12 V adapter, rests on folded cardboard secured with adhesive tape, which holds the test rig steady on the bench. The piezoelectric disc is bonded to the motor body and its output is routed to the conditioning stage on the breadboard. The MPU6050 is mounted on top of the motor housing (Figure 3.2). The ESP32, LEDs, buzzer, push button and conditioning circuit are assembled on the breadboard, while the SSD1306 OLED display and the relay module sit in front of it.

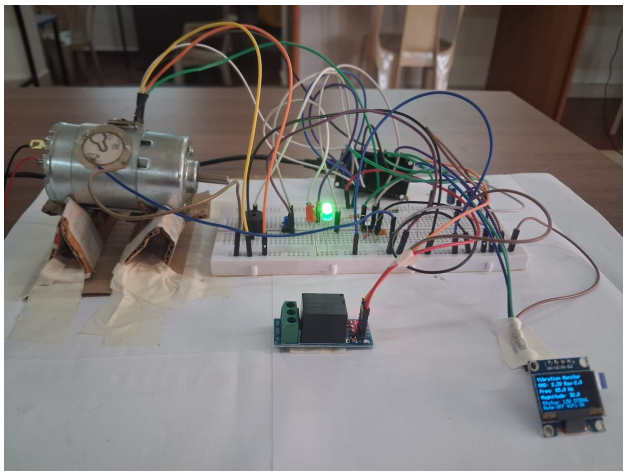


Figure 3.1. *Assembled prototype in normal operation — green LED on, OLED showing the LOW SIGNAL state.*

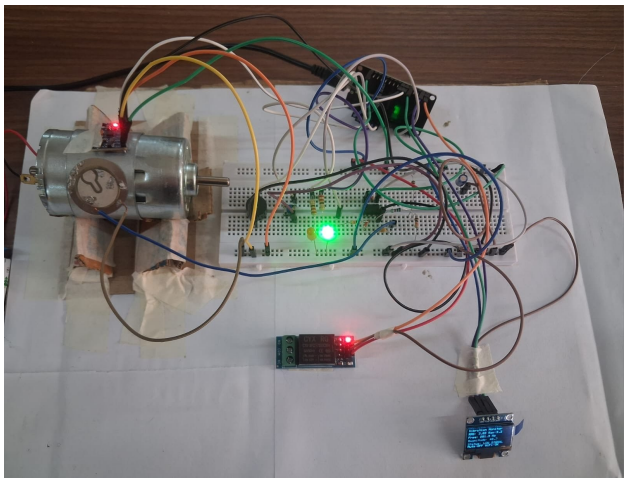


Figure 3.2. *Top view of the prototype showing the piezoelectric disc and the MPU6050 on the motor, the breadboard circuit, the relay module and the OLED.*

3.2 Pin and bus assignment

The OLED and the MPU6050 are placed on two separate I^2C buses so that the two peripherals do not share a bus. Table 3.1 lists the assignments used in the firmware.

Table 3.1: ESP32 pin and bus assignment

Signal	ESP32 pin	Notes
Piezo signal (conditioned)	GPIO34	ADC input, 12-bit (0 to 4095 counts)
Green LED	GPIO27	On in LOW SIGNAL
Red LED	GPIO14	On in WARNING and ALERT
Buzzer	GPIO26	Active buzzer, driven in ALERT unless muted
Push button	GPIO13	Internal pull-up, active low, 40 ms debounce
OLED SDA / SCL	GPIO21 / GPIO22	I^2C bus 0, address 0x3C
MPU6050 SDA / SCL	GPIO19 / GPIO18	Second I^2C bus at 100 kHz, address 0x68

3.3 Firmware architecture

The firmware is written in the Arduino framework for the ESP32 Dev Module and developed in the Arduino IDE, with the serial port at 115200 baud. It uses the Adafruit GFX, SSD1306, MPU6050 and Sensor libraries [10], the arduinoFFT library [9], and the WiFi and ThingSpeak libraries [5]. At start-up the firmware configures the 12-bit ADC and the output pins, initialises the OLED and shows a start-up message, initialises the MPU6050 (±2 g range, 94 Hz filter), and starts Wi-Fi in station mode. If the OLED or MPU6050 fails to initialise, the firmware halts and reports the fault on the serial port and display. Wi-Fi credentials and the ThingSpeak write key are stored as constants in the firmware and are not reproduced in this report.

The main loop repeats the following steps:

1. Check the mute button (debounced) and, if the Wi-Fi link is down, retry the connection every 15 s without blocking.
2. Acquire 128 piezo samples at 1 kHz using microsecond timing.
3. Remove the mean, compute the RMS and update the five-point moving average.
4. Apply the Hamming window, compute the FFT and find the dominant frequency and peak magnitude.
5. Acquire 128 MPU6050 samples at 100 Hz and compute the combined acceleration RMS.
6. Update the machine status and drive the LEDs and buzzer.
7. Refresh the OLED and print one line to the serial port.
8. Every 20 s, upload the five parameters to ThingSpeak.

One cycle takes about 1.4 s: the 128 ms piezo window plus the 1.28 s MPU6050 window, with a small overhead for display and serial output. Local monitoring continues if Wi-Fi is unavailable, because the upload step is skipped when the link is down.

3.4 Signal-processing parameters

Table 3.2: Signal-processing parameters

Parameter	Value	Parameter	Value
Piezo sampling frequency	1000 Hz (1 ms period)	Frequency resolution	7.8125 Hz
Block length	128 samples (128 ms)	RMS smoothing	Five-point moving average
ADC resolution	12 bits	MPU6050 sampling	100 Hz, 128 samples (1.28 s)
Window function	Hamming	MPU6050 range and filter	±2 g, 94 Hz low-pass
FFT bins searched	1 to 62 (7.8 Hz to about 484 Hz)	ThingSpeak upload interval	20 s

3.5 Status logic and output control

The status is updated once per cycle from the filtered piezo RMS. Table 3.3 gives the transition conditions.

Table 3.3: Status thresholds and hysteresis (filtered RMS in ADC counts; provisional values)

Transition	Condition
LOW SIGNAL → WARNING	RMS ≥ 6.0
LOW SIGNAL → ALERT	RMS ≥ 10.0
WARNING → ALERT	RMS ≥ 10.0
WARNING → LOW SIGNAL	RMS below 5.0
ALERT → WARNING	RMS below 8.5 and at least 5.0
ALERT → LOW SIGNAL	RMS below 5.0

Because the exit thresholds (5.0 and 8.5) are lower than the entry thresholds (6.0 and 10.0), a state is held while the RMS stays between the two values. This prevents rapid toggling of the LEDs and buzzer when the vibration level hovers near a limit.

Table 3.4: Output indication for each machine state

State	Green LED	Red LED	Buzzer
LOW SIGNAL	On	Off	Off
WARNING	Off	On	Off
ALERT	Off	On	On, unless muted

Each press of the push button toggles the buzzer mute; the OLED shows the mute state.

3.6 OLED display

The 128 × 64 OLED shows seven lines: the title, the filtered piezo RMS, the MPU6050 RMS in m/s^2, the dominant frequency, the peak magnitude, the machine status, and the mute and Wi-Fi flags (Wi-Fi: OK when connected).

3.7 ThingSpeak configuration

The five parameters are mapped to the channel fields in Table 3.5 and sent in a single write call; the HTTP response code is printed on the serial port for diagnosis.

Table 3.5: ThingSpeak field mapping

Field	Parameter
Field 1	Piezoelectric vibration RMS (filtered, ADC counts)
Field 2	Peak vibration magnitude (FFT)
Field 3	Machine status (0 = LOW SIGNAL, 1 = WARNING, 2 = ALERT)
Field 4	Dominant frequency (Hz)
Field 5	MPU6050 acceleration RMS (m/s^2)

4. Results and Discussion

4.1 Test setup

The prototype was run on the bench with the motor powered, while the OLED, the Arduino IDE Serial Monitor and the ThingSpeak dashboard were observed. The photographs and the serial capture presented here were taken during development with a build whose OLED shows the smoothed RMS (“RMS”) and the latest block RMS (“Raw”) and whose serial line does not yet include the MPU6050 value. The piezo processing in that build is the same as in the final firmware described in Chapter 3, which adds the MPU6050 RMS to the OLED, serial line and Field 5. Values quoted from photographs and charts are read off the images and are approximate.

4.2 Normal operation

In the normal condition (Figures 3.1 and 3.2) the filtered RMS stays in the low range, the status reads LOW SIGNAL, the green LED is on and the buzzer is silent. Typical OLED readings were a filtered RMS of about 2.4 to 3.0 counts, with the status LOW SIGNAL (Table 4.2).

4.3 Elevated vibration and alert operation

When the vibration level rose, the filtered RMS crossed the alert threshold and the system changed state. Figure 4.1 shows the prototype in the ALERT state: the OLED reports a filtered RMS of about 11 counts, which is above the 10.0 entry threshold, the red LED is lit and the buzzer is enabled (Mute: OFF).

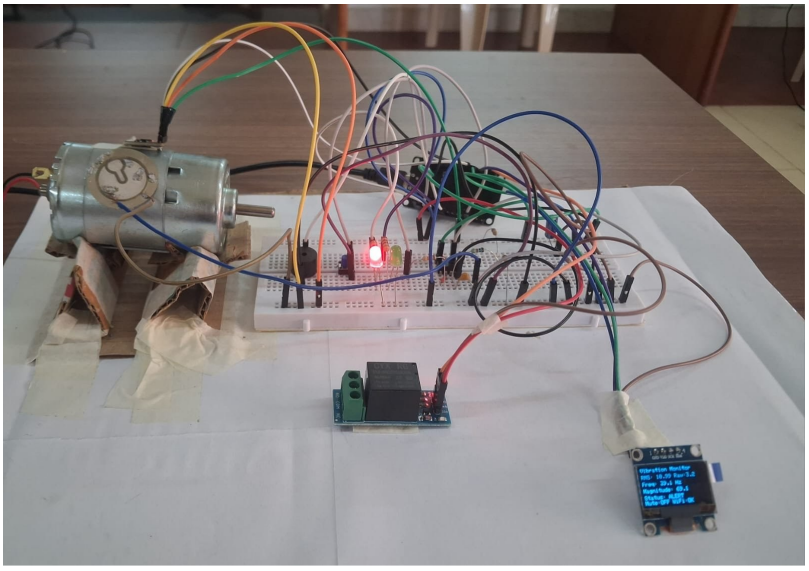


Figure 4.1. *Prototype in the ALERT state — red LED on, status ALERT on the OLED.*

In this photograph the latest block RMS ("Raw") is only about 3.2, while the smoothed value is about 11. This is consistent with the five-point average still containing larger bursts from the preceding cycles, and shows the smoothing and the 8.5 exit threshold keeping the alert active for a few cycles after a burst instead of dropping it on a single quiet block.

Table 4.2: OLED readings from the prototype photographs (approximate, read from the display)

Photograph	Filtered RMS	Latest block RMS	Dominant frequency	Peak magnitude	Status	LED
Figure 3.1	2.39	2.9	85.9 Hz	32.0	LOW SIGNAL	Green
Figure 3.2	3.03	3.3	281.3 Hz	48.7	LOW SIGNAL	Green
Figure 4.1	10.99	3.2	39.1 Hz	69.1	ALERT	Red

4.4 Serial Monitor log and hysteresis verification

Figure 4.2 shows the Serial Monitor output at 115200 baud, one line per cycle. Table 4.1 reproduces seven consecutive lines.

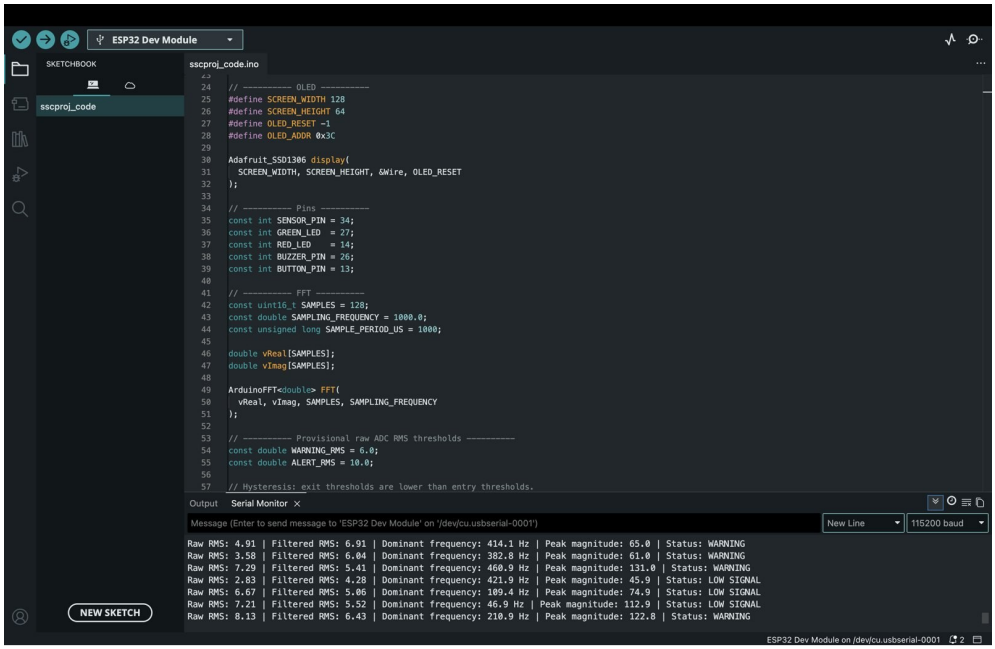


Figure 4.2. *Arduino IDE Serial Monitor output with the firmware source open in the editor.*

Table 4.1: Serial Monitor sample demonstrating hysteresis

Line	Raw RMS	Filtered RMS	Dominant frequency (Hz)	Peak magnitude	Status
1	4.91	6.91	414.1	65.0	WARNING
2	3.58	6.04	382.8	61.0	WARNING
3	7.29	5.41	460.9	131.0	WARNING
4	2.83	4.28	421.9	45.9	LOW SIGNAL
5	6.67	5.06	109.4	74.9	LOW SIGNAL
6	7.21	5.52	46.9	112.9	LOW SIGNAL
7	8.13	6.43	210.9	122.8	WARNING

The log confirms the intended behaviour of the status logic:

- **Lines 1 and 2:** the filtered RMS is above 6.0 and the status is WARNING.
- **Line 3:** the filtered RMS falls to 5.41, below the 6.0 entry threshold, yet the status remains WARNING because it has not dropped below the 5.0 exit threshold. This is the hysteresis.
- **Line 4:** the RMS falls to 4.28, below 5.0, and the status returns to LOW SIGNAL.
- **Lines 5 and 6:** the RMS (5.06 and 5.52) lies between 5.0 and 6.0 but the status stays LOW SIGNAL, because re-entering WARNING needs 6.0 or more.
- **Line 7:** the RMS reaches 6.43 and the status changes to WARNING.

The raw RMS varies between 2.83 and 8.13, a spread of about 5.3 counts, whereas the filtered RMS varies between 4.28 and 6.91, about 2.6 counts. The moving average therefore roughly halves the block-to-block variation seen by the classifier. The dominant frequencies are all multiples of the 7.8125 Hz bin width (for example 414.1 Hz is bin 53 and 46.9 Hz is bin 6), as expected for a 128-point FFT at 1 kHz.

4.5 ThingSpeak remote monitoring

Figure 4.3 shows the ThingSpeak dashboard with the first four fields plotted against time; Field 5 (MPU6050 acceleration RMS) is configured in the channel but is not part of this capture. The chart joins consecutive points with straight lines, so the long diagonal segments correspond to periods when the device was not powered or not uploading, not to measured changes. Data appear in three groups between about 10:20 and 11:50. Table 4.3 summarises what the charts show.

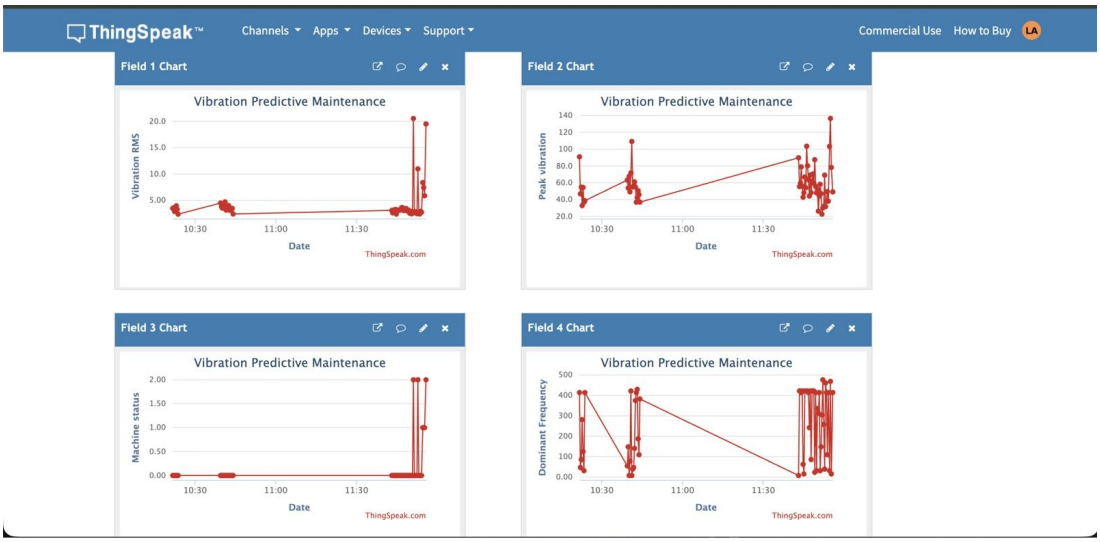


Figure 4.3. *ThingSpeak dashboard — Field 1 (vibration RMS), Field 2 (peak vibration), Field 3 (machine status) and Field 4 (dominant frequency).*

Table 4.3: Summary of ThingSpeak observations (approximate, read from the charts)

Field	Observation
Field 1: vibration RMS	About 2 to 5 counts for most of the record; excursions up to about 20 counts in the last group of readings, including intermediate values near 7 to 11
Field 2: peak vibration	About 30 to 110 in the first two groups; about 20 to 140 in the last group, with the highest values during the high-RMS period
Field 3: machine status	0 (LOW SIGNAL) for most of the record; 1 (WARNING) and 2 (ALERT) appear only in the last group, coinciding with the Field 1 excursions
Field 4: dominant frequency	Spread between 0 and about 480 Hz, with many readings clustered near 400 to 450 Hz

The status in Field 3 follows the vibration level in Field 1: ALERT readings (value 2) occur only where the RMS rises towards 10 counts and above. This confirms that the whole chain — sensing, processing, classification, Wi-Fi transmission and cloud logging — works end to end.

4.6 Discussion and limitations

The tests demonstrate a working sense, condition, process, indicate and report chain. The following limitations should be kept in view.

- **Provisional thresholds.** The 6.0 and 10.0 thresholds are in raw ADC counts of the conditioned piezo signal. They depend on the sensor mounting and the conditioning gain and have not been calibrated against known healthy and faulty states.
- **Level detection, not fault diagnosis.** The recorded tests show detection of raised vibration. A specific fault such as imbalance or bearing wear has not been injected and classified, so the spectral information is logged rather than used for decisions.
- **Scattered dominant frequency.** With a 7.8 Hz bin width and a motor that produces broadband vibration, the strongest bin changes from block to block (Field 4). A stable diagnostic frequency would need averaged spectra and a known running-speed reference. Unless the conditioning stage limits the bandwidth below

500 Hz, components above the Nyquist frequency can also alias into the spectrum.

- **Software-timed sampling.** The piezo samples are timed in software with analogRead, so timing jitter is possible, especially during Wi-Fi activity.
- **MPU6050 bandwidth.** At 100 Hz sampling the Nyquist frequency is 50 Hz, lower than the 94 Hz internal filter setting, so the acceleration RMS is a low-frequency complement to the piezo measurement.
- **Relay not yet used.** The relay module is installed but the firmware does not switch it, so motor cut-off is not part of the tested behaviour.

5. Conclusion

This project designed, built and tested an ESP32-based predictive maintenance prototype that monitors the vibration of a 775 DC motor. A piezoelectric disc with an op-amp conditioning stage and an MPU6050 accelerometer provide two complementary measurements. On the ESP32, the firmware computes the vibration RMS, a smoothed RMS, the dominant frequency and peak magnitude from a Hamming-windowed FFT, and the three-axis acceleration RMS.

All the objectives set in Section 1.3 were met at prototype level. The machine condition is classified into LOW SIGNAL, WARNING and ALERT with hysteresis, and the serial log confirms that the states hold and release at the designed thresholds. The condition is shown locally on an OLED, LEDs and a mutable buzzer, and five parameters are uploaded every 20 s to ThingSpeak, where the status trace followed the vibration RMS during the recorded excursions up to about 20 counts.

The work also illustrates the central ideas of the course: matching a high-impedance piezoelectric source to an ADC through signal conditioning, choosing sampling rate and block length to set the frequency range and resolution, and using averaging and hysteresis to turn a noisy signal into a stable decision. The thresholds are provisional and fault-specific diagnosis has not yet been demonstrated, so the system is best regarded as a dependable, low-cost foundation for condition-based monitoring rather than a finished diagnostic instrument.

6. Future Scope

- **Calibration and baselines.** Record vibration data for the motor in healthy and deliberately faulted states (for example an added imbalance weight) and set thresholds from this data, or compare the live spectrum with a stored healthy baseline as originally proposed.
- **Relay-based protection.** Enable the installed relay in firmware so that the motor is switched off automatically on a sustained ALERT, with a manual reset.
- **Better data acquisition.** Replace software-timed sampling with a hardware timer or the ESP32 I^2S/DMA interface, add an anti-aliasing filter, and use a higher sampling rate and longer records with averaged spectra.
- **Use of the MPU6050 spectrum.** Run an FFT on the accelerometer data and fuse it with the piezo measurement for more reliable decisions.
- **Fault classification.** Build a labelled vibration dataset and apply machine-learning methods to classify imbalance, misalignment and bearing faults.
- **More parameters.** Add temperature, motor current and rotational speed sensing to relate vibration to operating conditions.
- **Alerts and dashboards.** Use ThingSpeak's alerting features for notifications and add trend and threshold analysis on the cloud side.
- **Packaging and scale-up.** Move the circuit to a PCB with an enclosure and extend the design to several monitored machines on one network.

Bibliography

[1] R. B. Randall, *Vibration-based Condition Monitoring: Industrial, Aerospace and Automotive Applications*. Chichester, U.K.: Wiley, 2011.

[2] R. K. Mobley, *An Introduction to Predictive Maintenance*, 2nd ed. Amsterdam, The Netherlands: Butterworth-Heinemann, 2002.

[3] InvenSense Inc., "MPU-6000 and MPU-6050 Product Specification," Datasheet, Rev. 3.4.

[4] Espressif Systems, "ESP32 Series Datasheet" and "ESP32 Technical Reference Manual."

[5] MathWorks, "ThingSpeak IoT Analytics Platform — Documentation," thingspeak.com.

[6] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2010.

[7] J. Fraden, *Handbook of Modern Sensors: Physics, Designs, and Applications*, 5th ed. Cham, Switzerland: Springer, 2016.

[8] S. Franco, *Design with Operational Amplifiers and Analog Integrated Circuits*, 4th ed. New York, NY, USA: McGraw-Hill, 2015.

[9] E. Condes, "arduinoFFT: FFT library for Arduino," open-source software library.

[10] Adafruit Industries, "Adafruit GFX, SSD1306, MPU6050 and Unified Sensor libraries," open-source software libraries.

[11] Course material, BAEIE102 — Sensors and Signal Conditioning, Dr. S. Vivekanandan, School of Electronics Engineering.

[12] Analog Devices Inc., "ADXL345: 3-Axis, ±2 g/±4 g/±8 g/±16 g Digital Accelerometer," Datasheet.